

Справка по решению задачи Р3 «Пробуждение Квизац Хадерача»

Идея алгоритма

Требуется реализовать поразрядную сортировку **RADIX SORT** по основанию 256 (по байтам) для n целых 32-битных чисел ($n \leq 10^6$).

Используется вариант **LSD Radix Sort** (от младшего разряда к старшему):

- Число рассматривается как 4 байта: b_0, b_1, b_2, b_3 (от младшего к старшему).
- Выполняется 4 прохода по массиву: на проходе p ($p = 0, 1, 2, 3$) сортируем числа по байту b_p .
- На каждом проходе используется **устойчивый counting sort** с основанием 256.

Благодаря устойчивости промежуточных сортировок, после обработки всех четырёх байтов массив оказывается полностью отсортирован.

Сортировка по одному байту

Для фиксированного байта b_p :

1. Обнуляем массив частот `cnt[256]`.
2. Для каждого элемента x :
 - интерпретируем его как беззнаковое `uint32_t`;
 - извлекаем байт: $byte = (x \gg (8p)) \& 255$;
 - увеличиваем `cnt[byte]`.
3. Строим префиксные суммы в `cnt`: после этого `cnt[v]` содержит начальный индекс для корзины с байтом v .
4. Вторым проходом по массиву для каждого элемента x :
 - ещё раз вычисляем $byte$;
 - помещаем x во временный массив b по индексу `cnt[byte]` и увеличиваем `cnt[byte]`.
5. Меняем местами роли массивов: $a \leftrightarrow b$.

Такой **counting sort** является стабильным, потому что элементы обрабатываются во втором проходе в исходном порядке и каждый раз используются возрастающие индексы.

Обработка отрицательных чисел

Целые числа в C++ хранятся в формате дополнительного кода. Если просто сортировать по старшему байту b_3 как по беззнаковому, то отрицательные числа (у которых старший бит равен 1) окажутся в конце массива.

Чтобы обеспечить правильный порядок (отрицательные $<$ неотрицательных), на **последнем проходе** ($p = 3$) используется преобразование:

$$byte' = byte \oplus 0x80,$$

то есть инвертируется самый старший бит байта b_3 . Это эквивалентно циклическому сдвигу диапазона так, что все отрицательные числа попадают в “младшую” половину корзины ($0 \dots 127$), а неотрицательные — в “старшую” ($128 \dots 255$). В результате после последнего прохода массив оказывается отсортированным с учётом знака.

Ограничения по времени и памяти

- Число проходов: 4 (по числу байтов в `int32_t`).
- Время работы: каждый проход — $O(n)$, итого $O(4n) = O(n)$ при фиксированном числе байтов.
- Память:
 - два массива по n элементов (основной и временный);
 - массив счётчиков `cnt[256]`.

При $n \leq 10^6$ и размере элемента 4 байта суммарная память $\approx 2 \cdot 4n + 256 \cdot 4$ байт $\ll 256$ МБ, то есть решение укладывается в лимит.

Замечания по реализации

- Используется только собственная реализация `RADIX SORT` и `counting sort`; стандартные сортировки (`std::sort` и т. п.) не применяются.
- Основание 256 (байты) позволяет эффективно извлекать разряды с помощью побитового сдвига и маски `& 255`, без операций деления и взятия по модулю.
- Для ускорения ввода–вывода включены:

```
std::ios::sync_with_stdio(false);  
std::cin.tie(nullptr);
```